
The Model & Basic Computations

Chapter 1 and 2

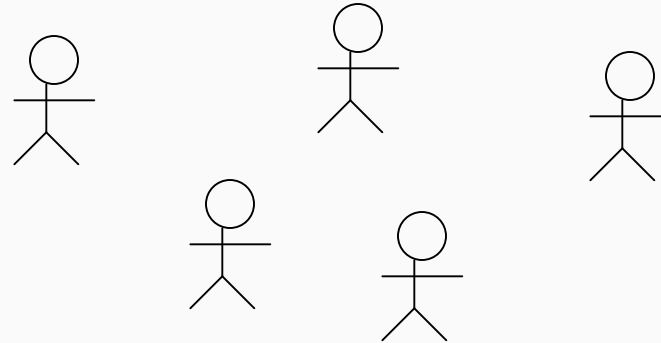
The Model

Broadcast

Wake-up

Distributed Environment

Multiplicity



Autonomy



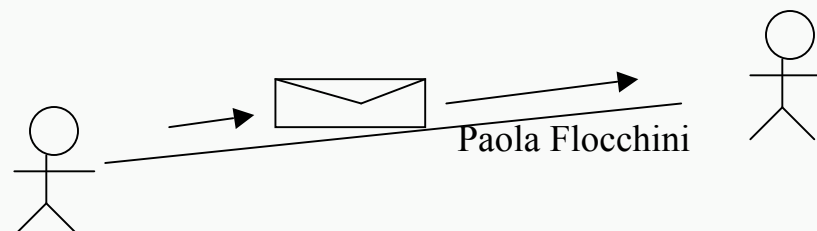
clock



memory

computing capabilities

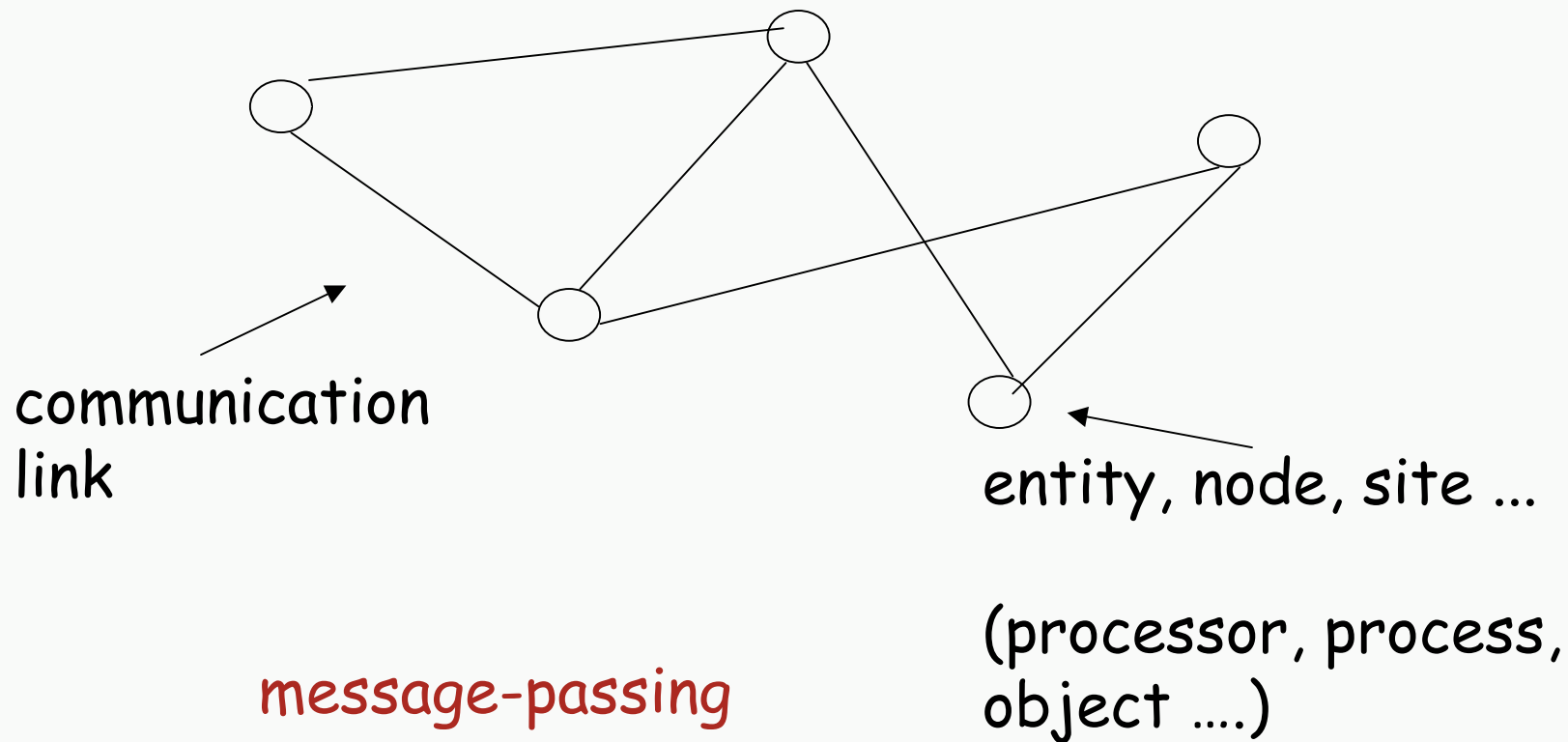
Interaction

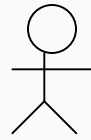


typically by
exchange of messages

The Model

Collection of entities that communicate
by exchanging messages





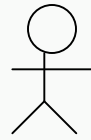
Entity x

In memory:

registers
state(x) value(x)

Possible operations:

- local storage and processing
- transmission of messages 
- (re)setting the clock 
- changing the value of the registers



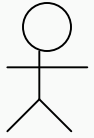
Entity x

state(x)

Finite set of possible system states
(ex: {idle, computing, waiting, processing})

Always defined

At any time an entity is in one of these states



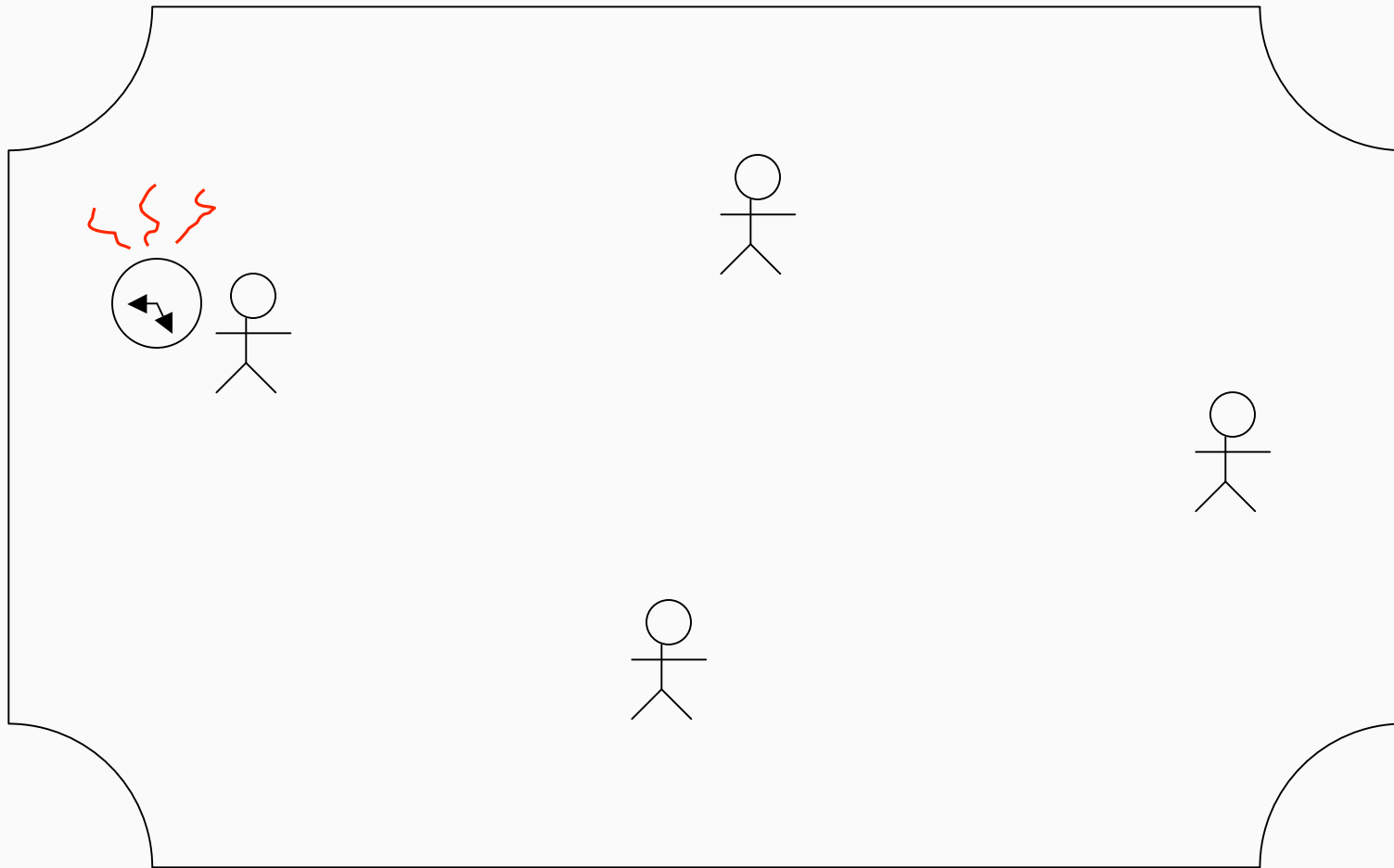
External Events

The behavior of an entity is reactive:
triggered by events

Possible events:

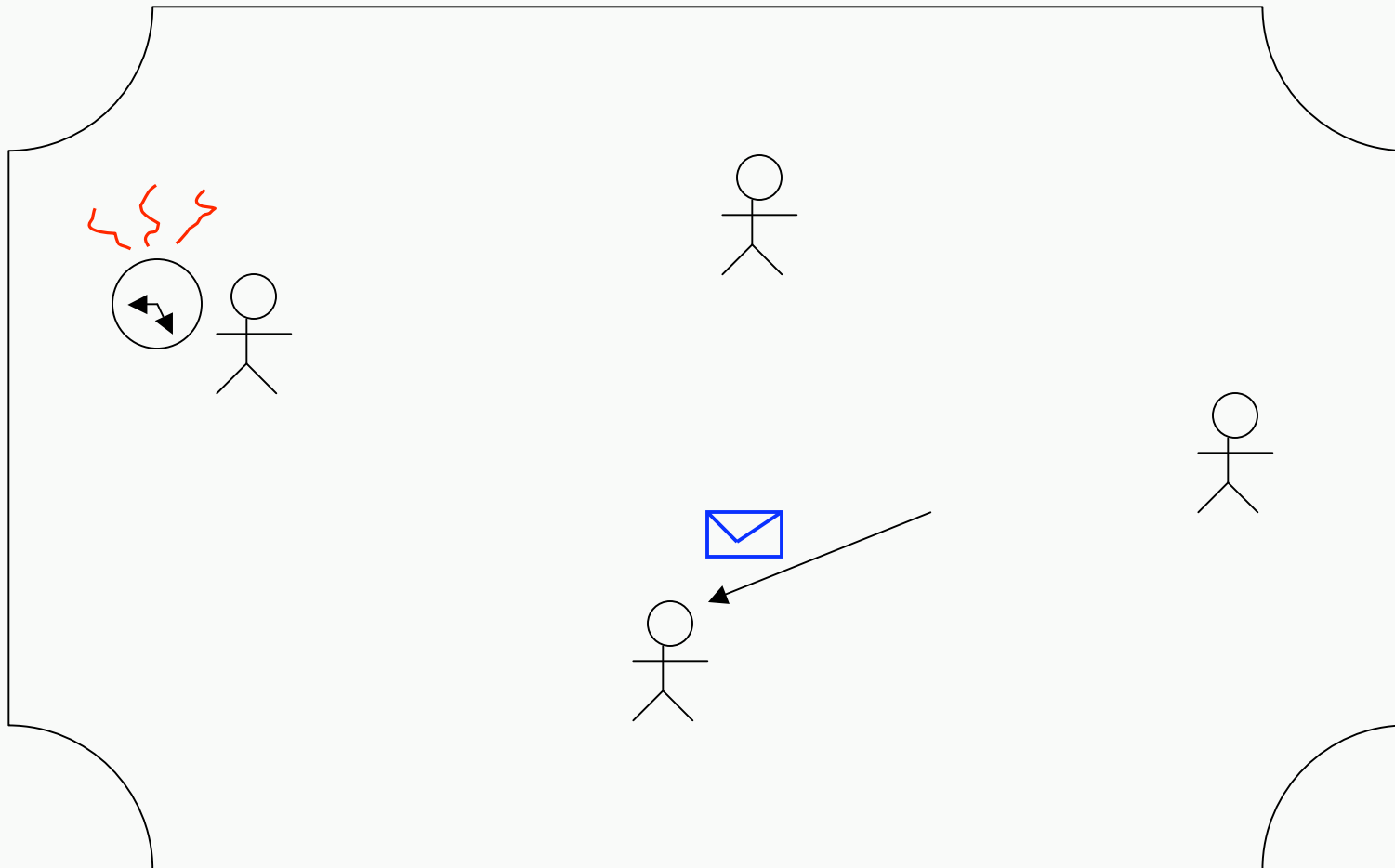
- clock tick
- receiving a message
- spontaneous impulse

clock ring



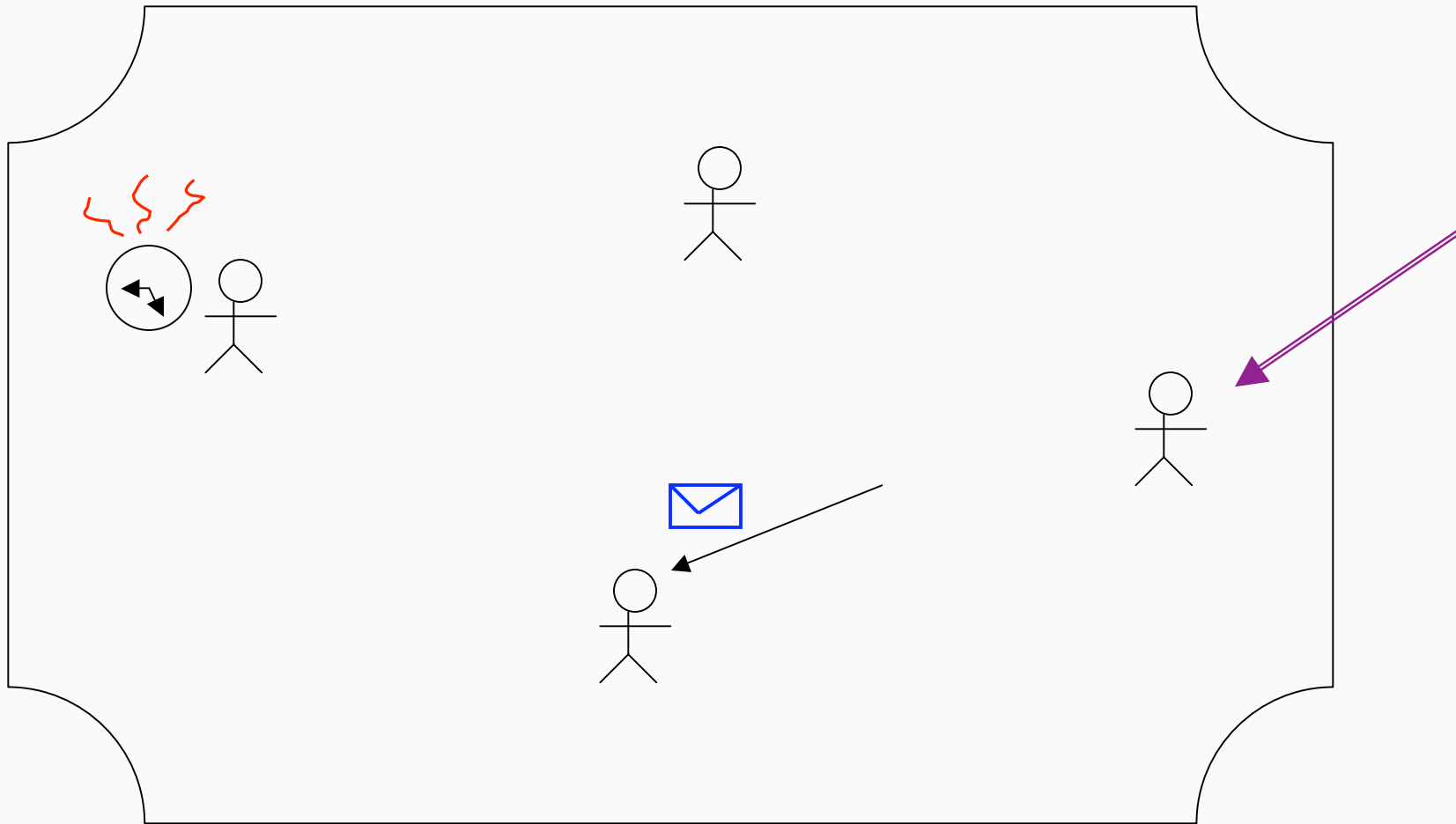
Paola Flocchini

message arrival



Paola Flocchini

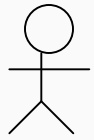
spontaneous impulse



Paola Flocchini

The reaction of an entity
depends on the event and on its state

State × Event → Action



Actions

Action: sequence of activities, e.g.,

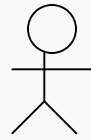
{
 computing
 sending message
 change state

an action is **atomic**

the activities cannot be interrupted

an action is **terminating**

the activities must terminate within finite time



Entity Behavior

Rule

State x Event $\longrightarrow$ Action

Behavior $B(x)$ = set of **rules** of entity x for all possible events and all possible states

The algorithm
The protocol

DETERMINISTIC

(state, event) $\rightarrow$ only ONE action

COMPLETE

($\forall$ (state, event) $\exists$ an action)

System Behavior

$$B = \{ B(x) : x \in E \}$$

A system is SYMMETRIC (or homogeneous)
if all the entities have the same behavior

$$B(x) = B(y), \forall x, y \in E$$

Property: Every system can be made symmetric

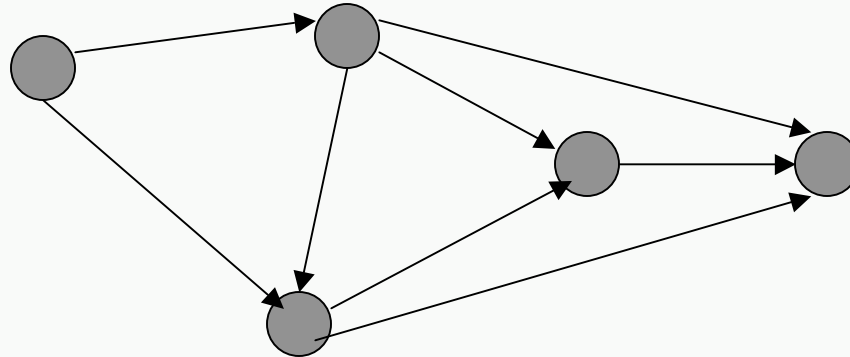


```
s x e → If role = workstation
          ActionW(s,e)
        else
          ActionS(s,e)
```

Communication

Message: finite sequence of bits

Communication Network:



Paola Flocchini

Point-to-point Model

$N_i(x)$ = in-neighbors of entity x

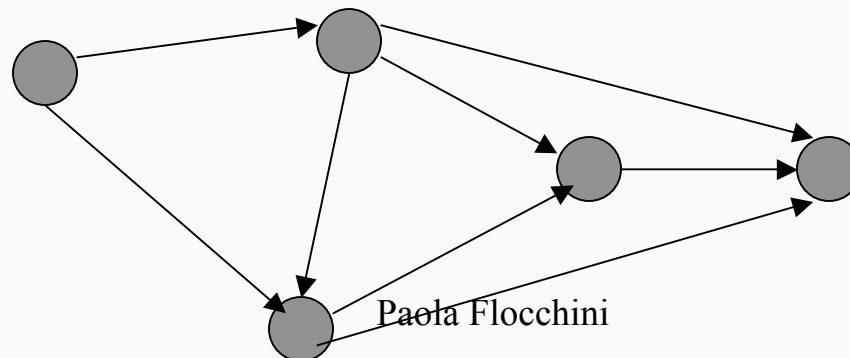
```

graph LR
    y((y)) --> x((x))
    z((z)) --> x
    x --> w((w))

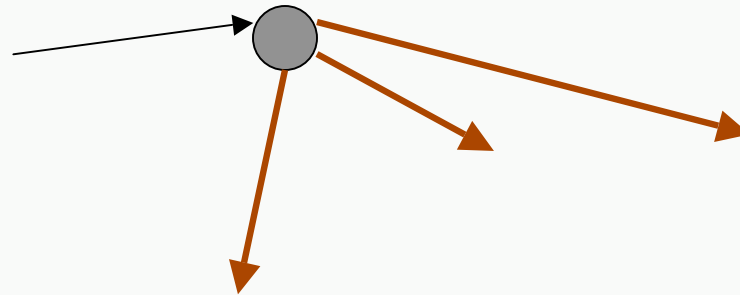
```

$$G = (V, A)$$

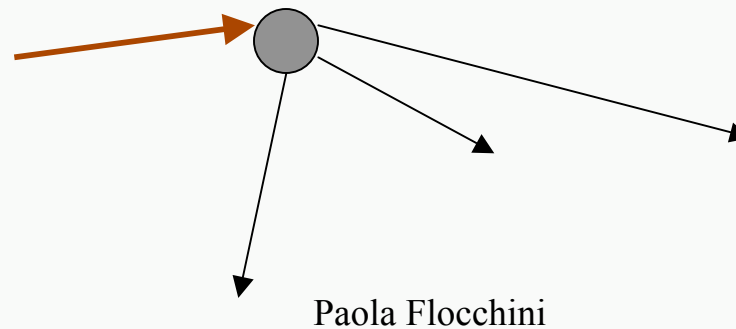
A: Arcs defined by N



An entity x can send a message only to its out-neighbors $N_o(x)$



and receive from the in-neighbours $N_i(x)$



Axioms

Finite Transmission Delays

In absence of faults a message from x to its out-neighbour y reaches y in finite time

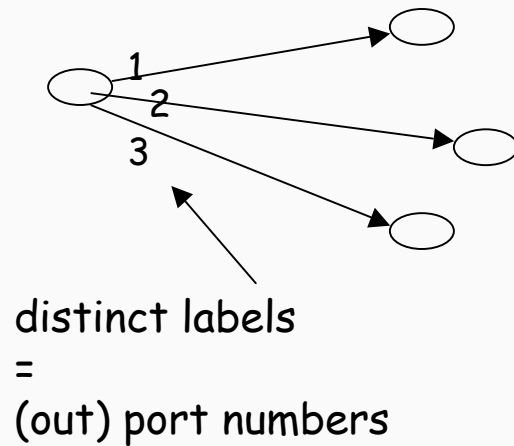


Local orientation

Each entity distinguishes among its neighbors

Local orientation: more precisely

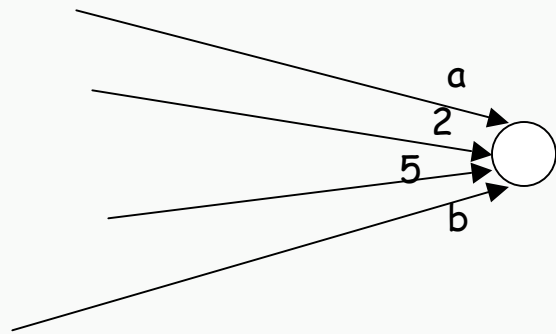
Each entity distinguishes among its out-neighbors



Send Message **to** 3

Local orientation: more precisely

Each entity distinguishes among its in-neighbors

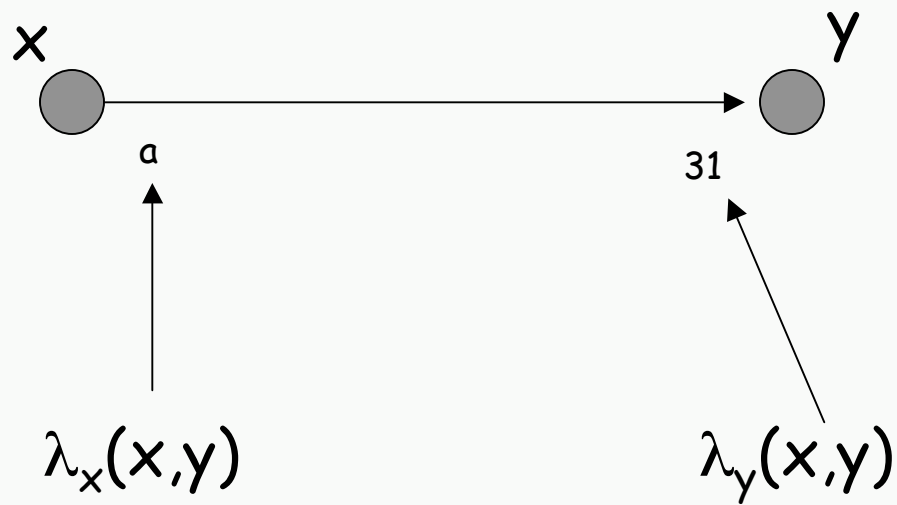


distinct labels
=
(in) port numbers

When a message arrives, the entity can detect from which port

Local orientation: more precisely

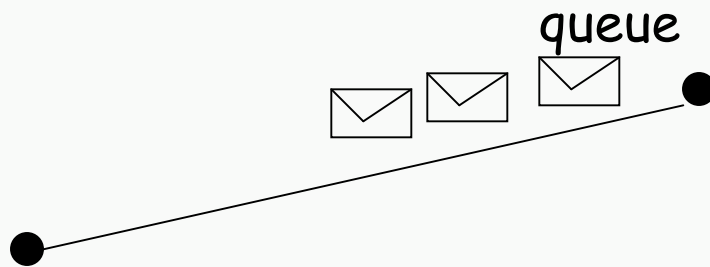
for an edge (x,y) there are two labels:



Topology = labeled graph (G, λ)

Restrictions of the model: examples

Communication Restrictions



Message Ordering

(**FIFO**)

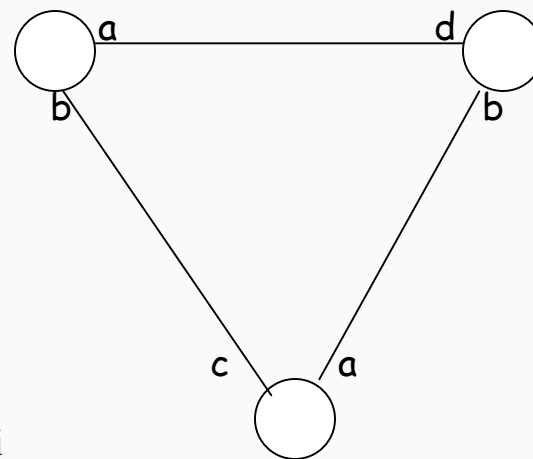
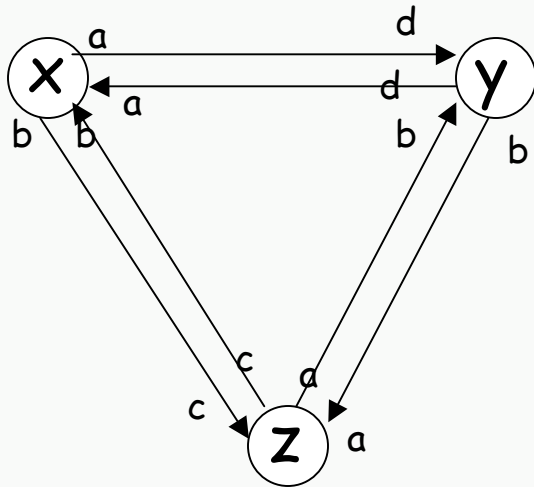
In absence of failures, msgs transmitted along the same link arrive in the same order.

Restrictions of the model: examples

Communication Restrictions

Bidirectional Links

$$\forall x, N_i(x) = N_o(x) = N(x) \text{ and} \\ \forall y \in N(x): \lambda_x(x,y) = \lambda_x(y,x)$$



Paola Flocchini

Restrictions of the model: examples

Reliability Restrictions:

1. *Guaranteed delivery:*
Any message that is sent will be received uncorrupted
2. *Partial Reliability:*
There will be no failures
3. *Total Reliability:*
No failures have occurred nor will occur

.....

Restrictions of the model: examples

Topological restriction:

The graph G is strongly connected

.....

Knowledge Restrictions

Knowledge of number of nodes
Knowledge of number of links
Knowledge of diameter

.....

Restrictions of the model: examples

Time restriction:

Bounded Communication Delay:

There exists a constant Δ such that, in absence of failures, the communication delay of any message on any link is at most Δ

Synchronized clocks:

All local clocks are incremented by one unit simultaneously and interval are constant

.....

Paola Flocchini

Complexity measures - Performance

1. Amount of communication

number of messages exchanged
(finer granularity: number of bits)



point of view
of SYSTEM

2. Time

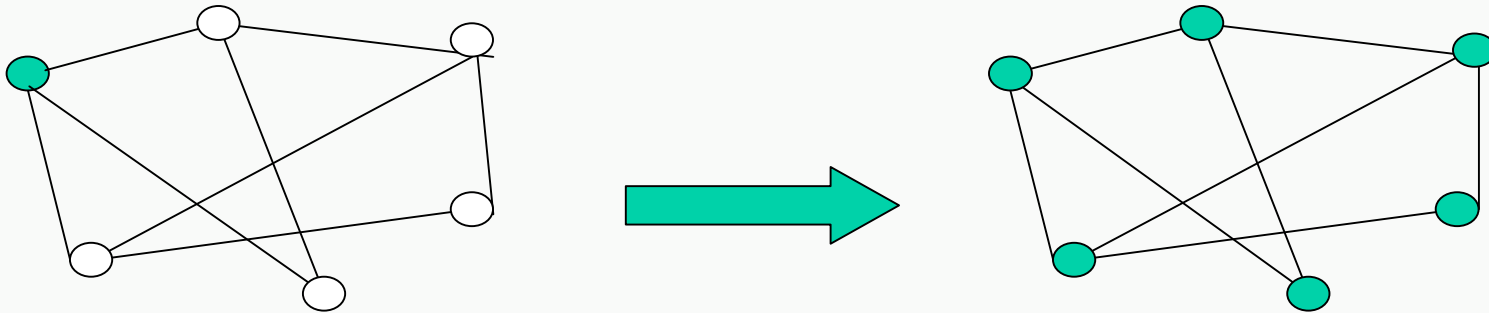
point of view
of USER

Communication delays are in general
unpredictable !!!

Ideal time:

1 unit of time to transmit 1 message

Example - Broadcast



Assumptions = Restrictions

Unique Initiator
Total reliability
Bidirectional links
G is connected

By definition of problem

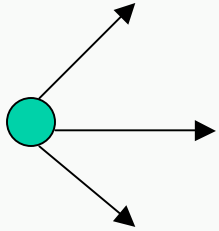
Simplifying assumptions

Otherwise unsolvable

Algorithm FLOOD

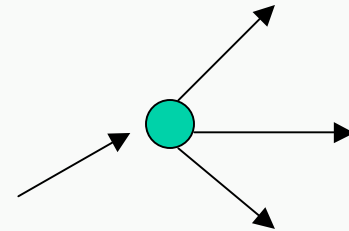
The idea: If an entity knows something, it sends the info to its neighbours

One entity is INITIATOR, the others are SLEEPING



INITIATOR
spontaneously
send(I) to N(x)

SLEEPING
receiving(I)
send(I) to N(x)



The idea: If an entity knows something, it sends
it to its neighbours **except the sender**

```
INITIATOR  
spontaneously  
send(I) to N(x)
```

```
SLEEPING  
receiving(I)  
send(I) to N(x) - {sender}
```

It does not terminate

Paola Flocchini

$S = \{\text{initiator}, \text{sleeping}, \text{done}\}$

Algorithm for node x:

```
INITIATOR  
spontaneously  
    send(I) to N(x)  
    become(DONE)
```

```
SLEEPING  
receiving(I)  
    send(I) to N(x) - {sender}  
    become(DONE)
```

Algorithm FLOOD

Algorithm for node x:

```
INITIATOR  
spontaneously  
    send(I) to N(x)  
    become(DONE)
```

```
SLEEPING  
receiving(I)  
    send(I) to N(x) - {sender}  
    become(DONE)
```

```
DONE
```

Algorithm FLOOD

Algorithm for node x:

State

```
graph TD; State[State] --> Init[If INITIATOR]; State --> Sleep[If SLEEPING]; State --> Done[If DONE]; Init --> InitBody["spontaneously<br/>send(I) to N(x)<br/>become(DONE)"]; Sleep --> SleepBody["receiving(I)<br/>send(I) to N(x) - {sender}<br/>become(DONE)"]; Done --> DoneBody["do-nothing"];
```

If **INITIATOR**
spontaneously
send(I) to N(x)
become(DONE)

If **SLEEPING**
receiving(I)
send(I) to N(x) - {sender}
become(DONE)

If **DONE**
do-nothing

Algorithm for node x:

If **INITIATOR**
spontaneously

send(I) to N(x)
become(DONE)

If **SLEEPING**
receiving(I)

send(I) to N(x) - {sender}
become(DONE)

If **DONE**
do-nothing

Event

Algorithm for node x:

If INITIATOR

spontaneously

send(I) to N(x)
become(DONE)

If SLEEPING

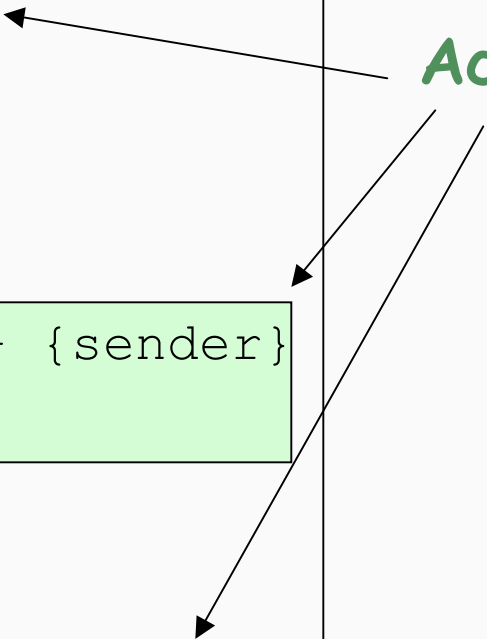
receiving(I)

send(I) to N(x) - {sender}
become(DONE)

If DONE

do-nothing

Action



Example

Correctness

The Algorithm terminates in finite time

It follows from: G connected and total reliability

Termination

Local Termination: when DONE

Global Termination: when?

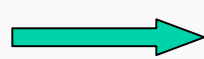
Message complexity

Worst Case

m = number of links

Worst for all possible initiators
and for all possible executions

Messages: ≤ 2 on each link

 $\leq 2m$

$O(m)$

More precisely:

Let s be the initiator

$$|N(s)| + \sum_{x \neq s} (|N(x)| - 1)$$

$$= \sum_x |N(x)| - \sum_{x \neq s} 1$$

$$\sum_x |N(x)| = 2m$$

Paola Flocchini

$$2m - (n-1)$$

Time Complexity - Ideal Time

Worst Case

Worst for all possible initiators
and for all possible executions

Time: (ideal time)

$$\begin{aligned} \text{Max}_{x\} \{d(x,s)\} &= \text{eccentricity of } s \\ &\leq \text{Diameter}(G) \leq n-1 \end{aligned}$$

$O(n)$

Time and Events

External events:

spontaneously
receiving
when (clock)

Actions may generate events

send	generates	<i>receiving</i>
set-clock	generates	<i>when</i>

Generated events might not occur (in case of faults).
If they occur, they occur **later**.

In the case of *receiving* with some unpredictable delay.

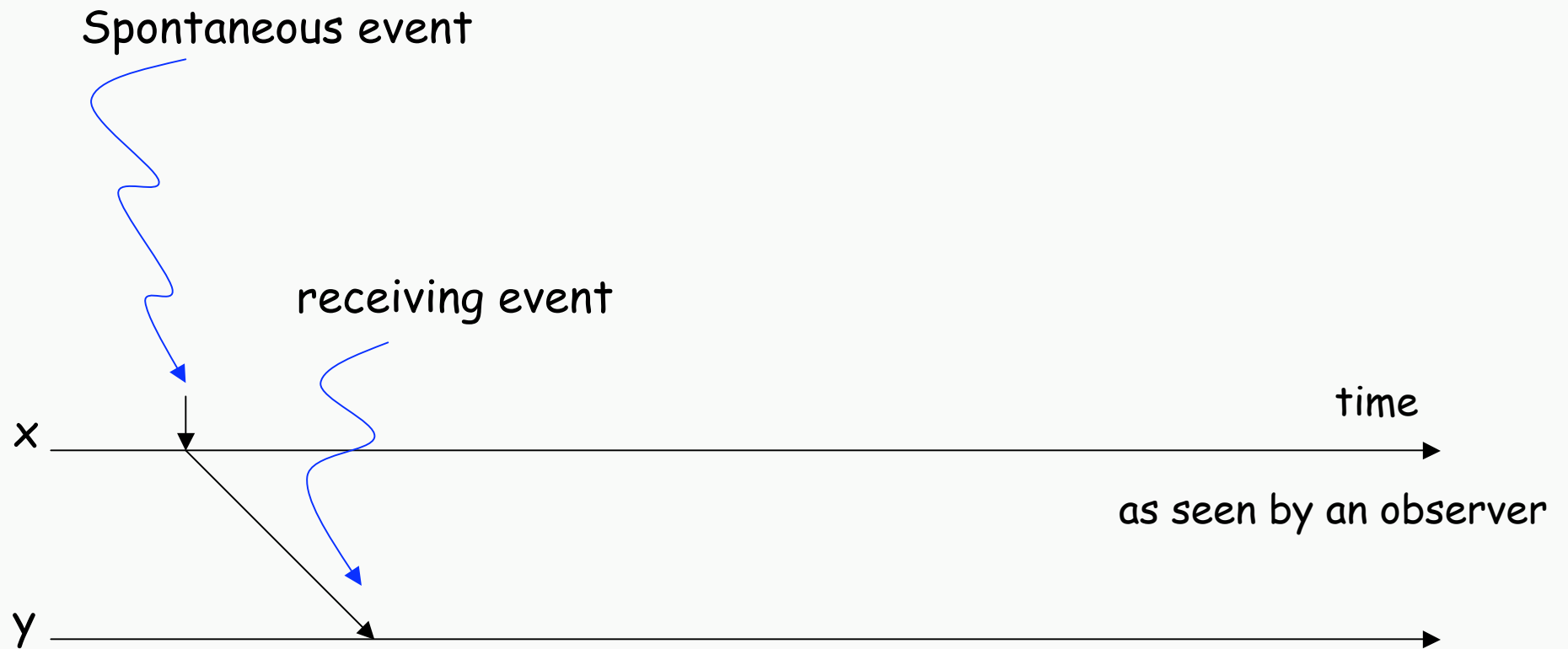
Different delays ---> different executions

Different executions could have different outcomes

(Spontaneous events are considered generated
before execution starts: initial events)

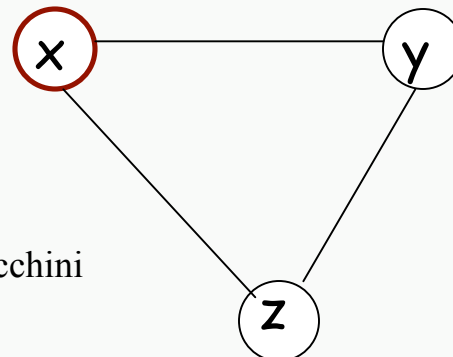
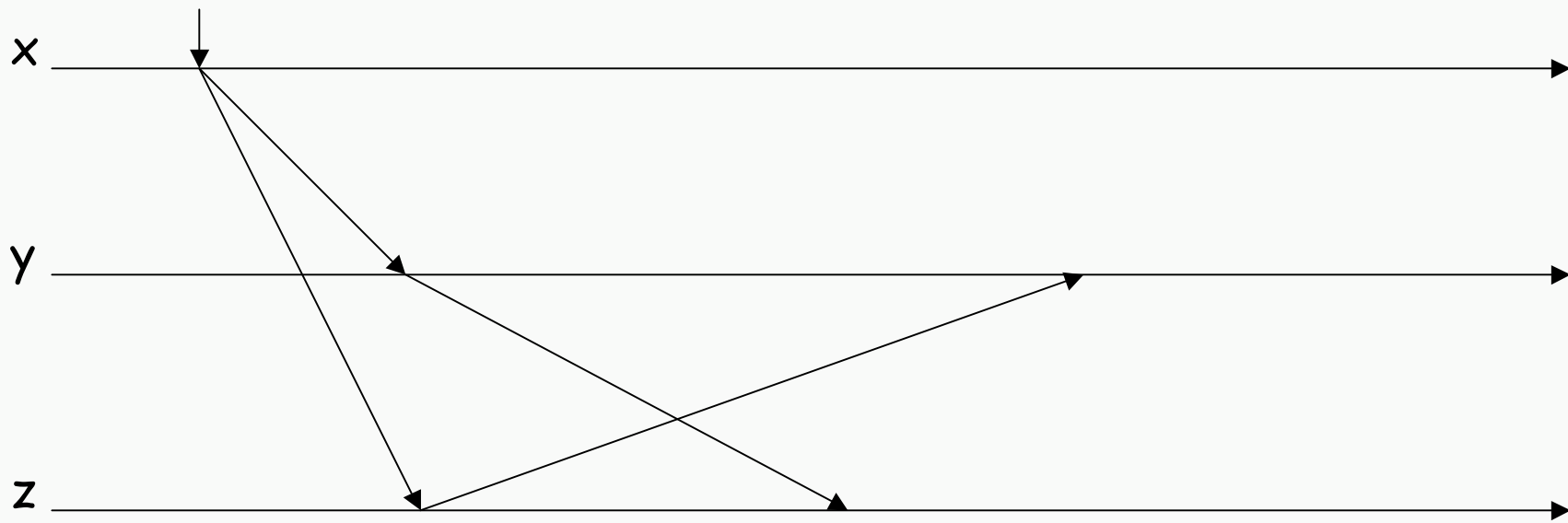
An execution is fully described by the sequence of events
that have occurred

Time x Event diagram



Example: Time x Event diagram of Flooding

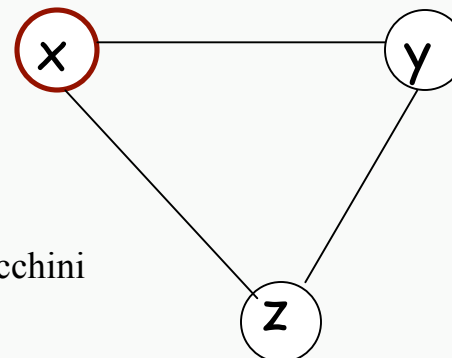
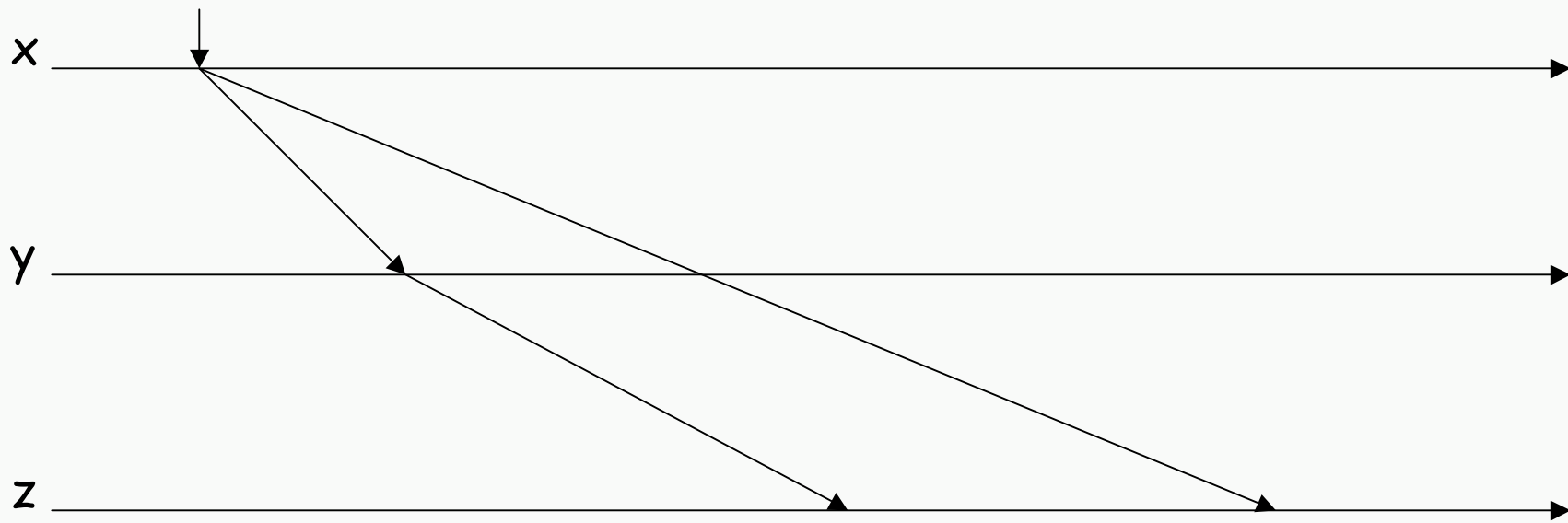
One possible execution



Paola Flocchini

Example: Time x Event diagram of Flooding

Another execution



Paola Flocchini

Knowledge

P = fact; x = entity; S = set of entities.

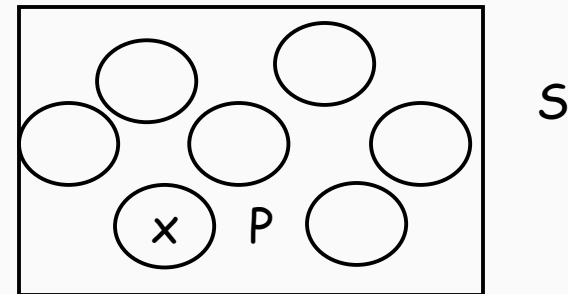
- **Local knowledge LK**

$P \in LK(x)$.



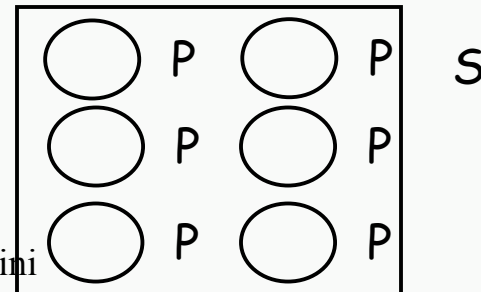
- **Implicit knowledge IK**

$P \in IK(S)$ if $\exists x \in S: P \in LK(x)$.



- **Explicit knowledge EK**

$P \in EK(S)$ if $\forall x \in S: P \in LK(x)$.



• Common knowledge CK

$P \in CK(S)$ if

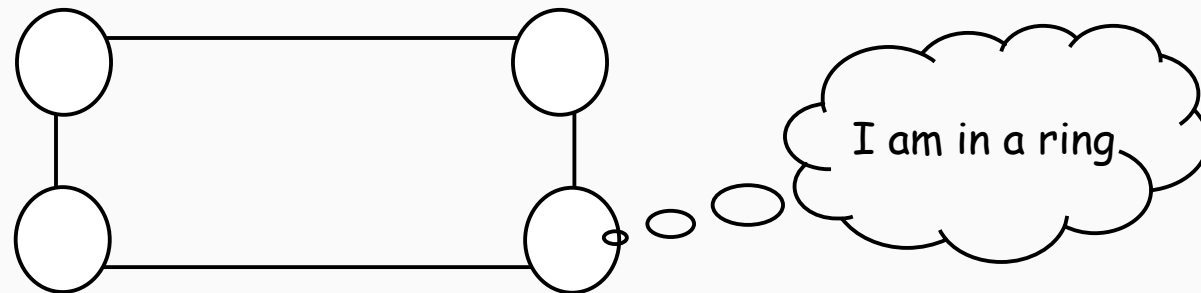
$\forall x \in S, P \in LK(x) \wedge$

$\forall x \in S (\forall x \in S, P \in LK(x)) \in LK(x) \wedge$

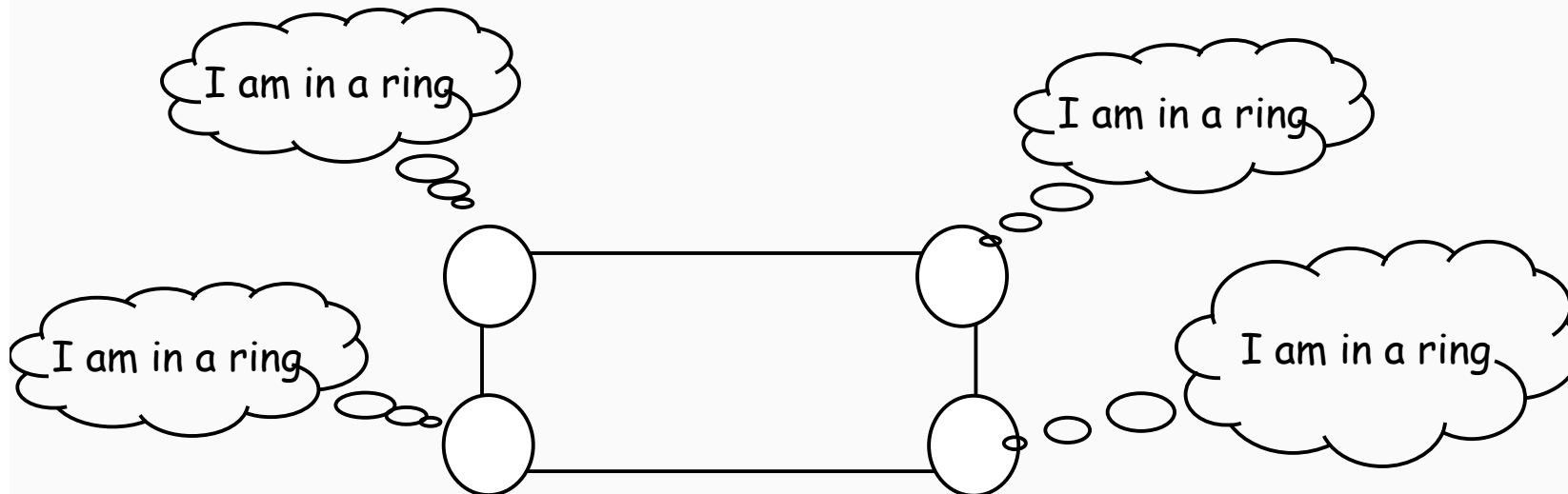
$\forall x \in S ((\forall x \in S, P \in LK(x)) \in LK(x)) \in LK(x) \wedge \dots$

Examples

Implicit knowledge



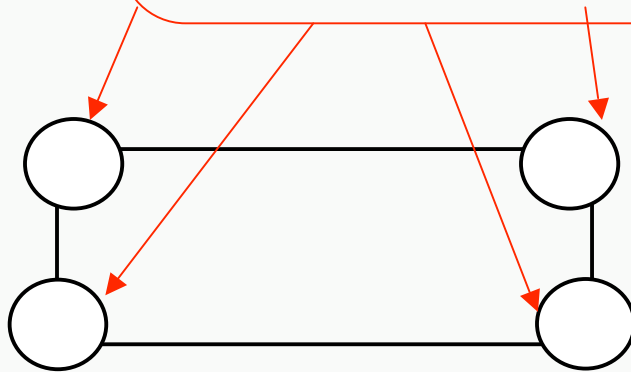
Explicit knowledge

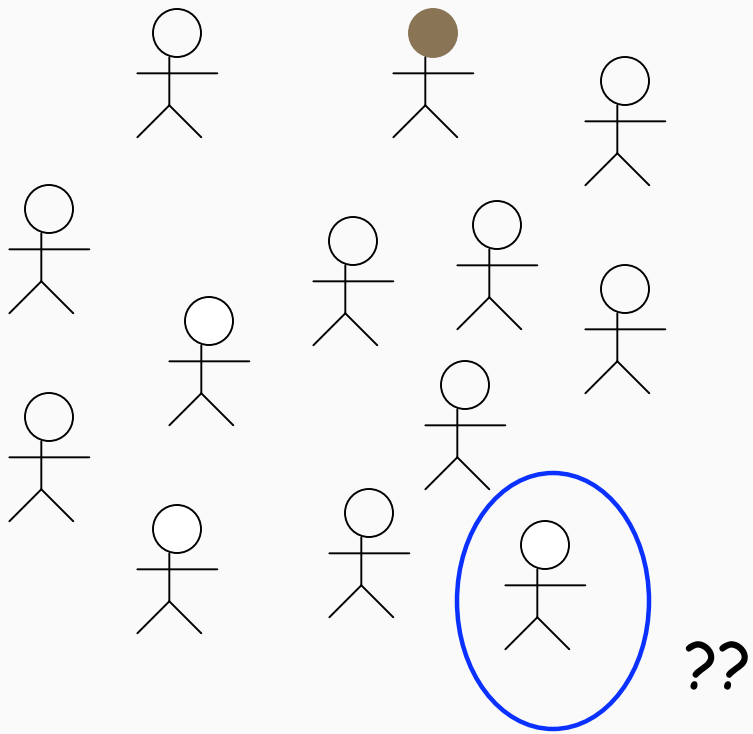


Paola Flocchini

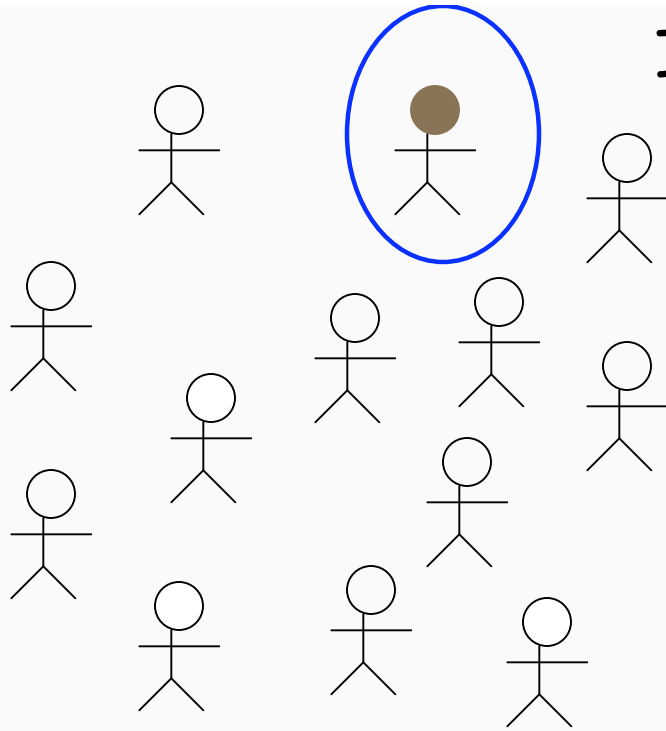
Common knowledge

I know I'm in a ring,
everyone knows it,
everyone knows that
everyone knows it, ...
....



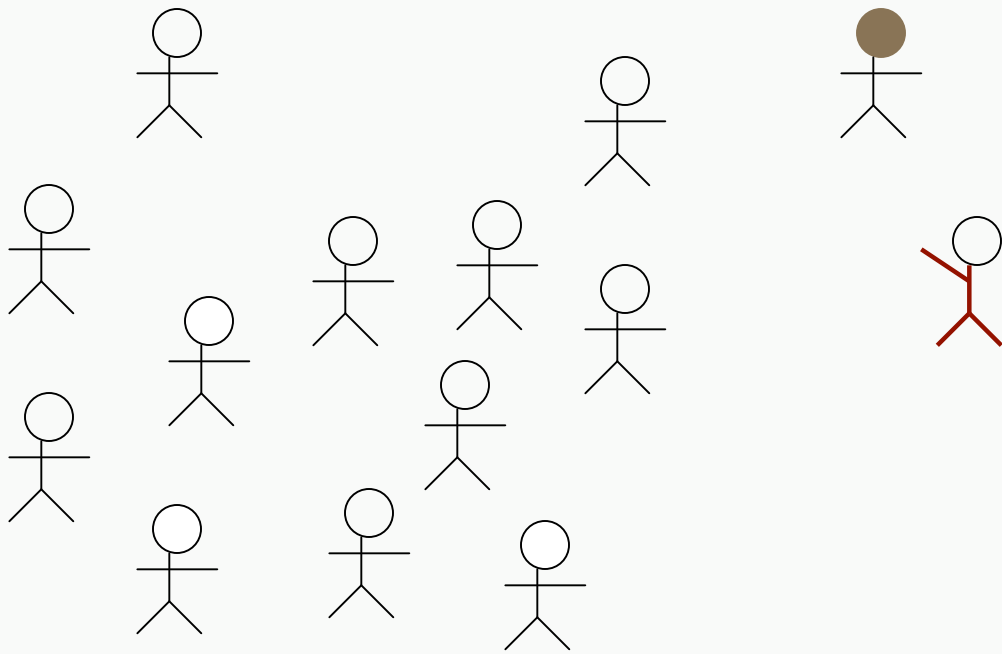


I see one dirty forehead

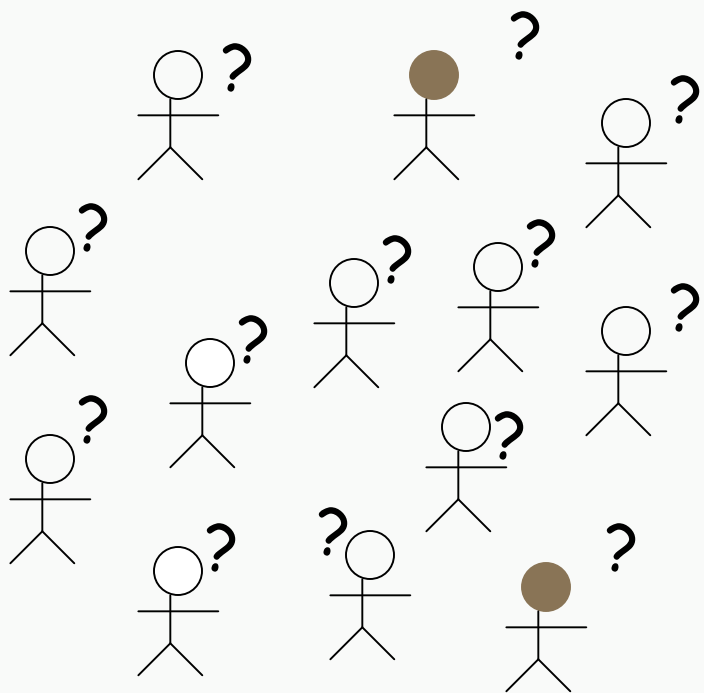


I see everybody else is clean

I must be dirty !!!!!

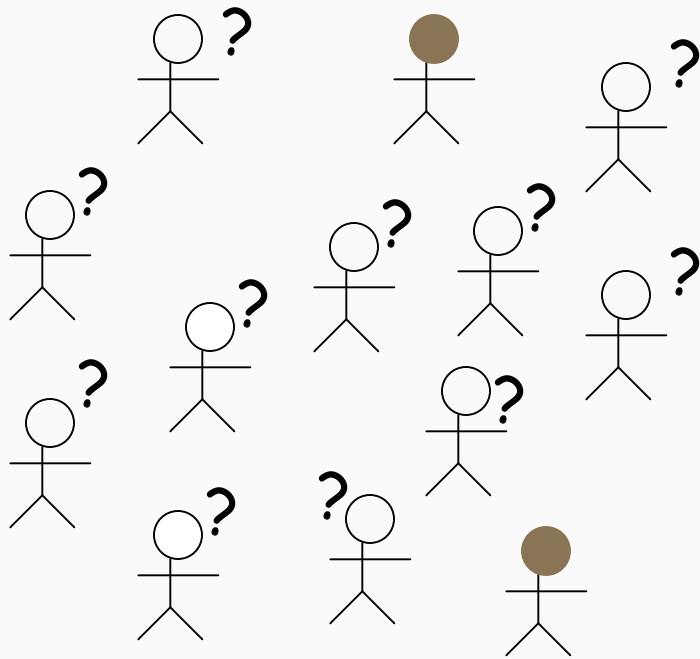


$t=1$



$t=1$

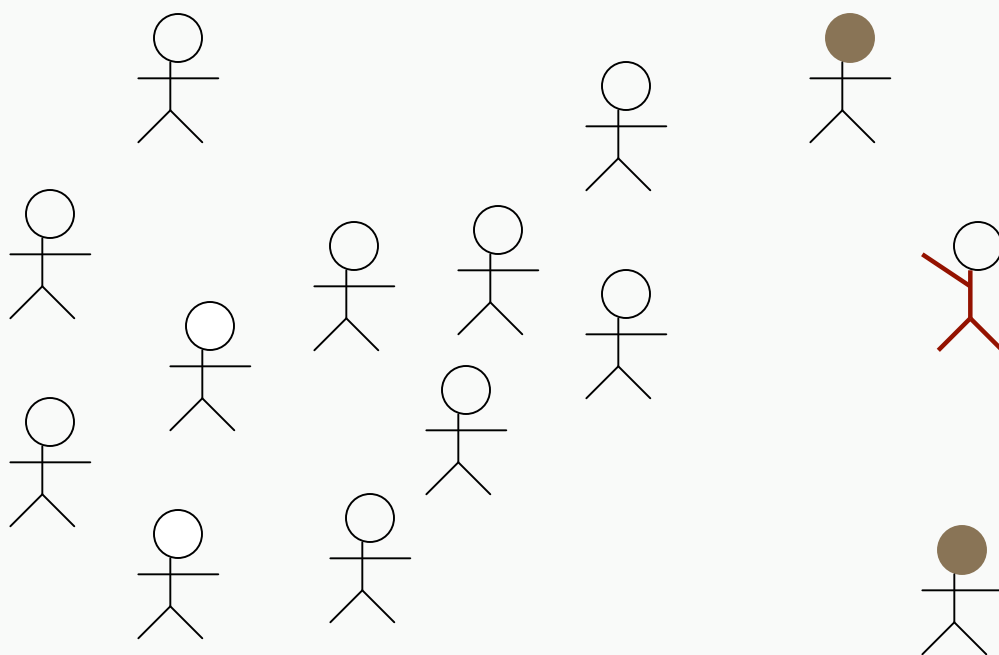
The other didn't go forward ... I must be dirty too !!!!



$t=1$

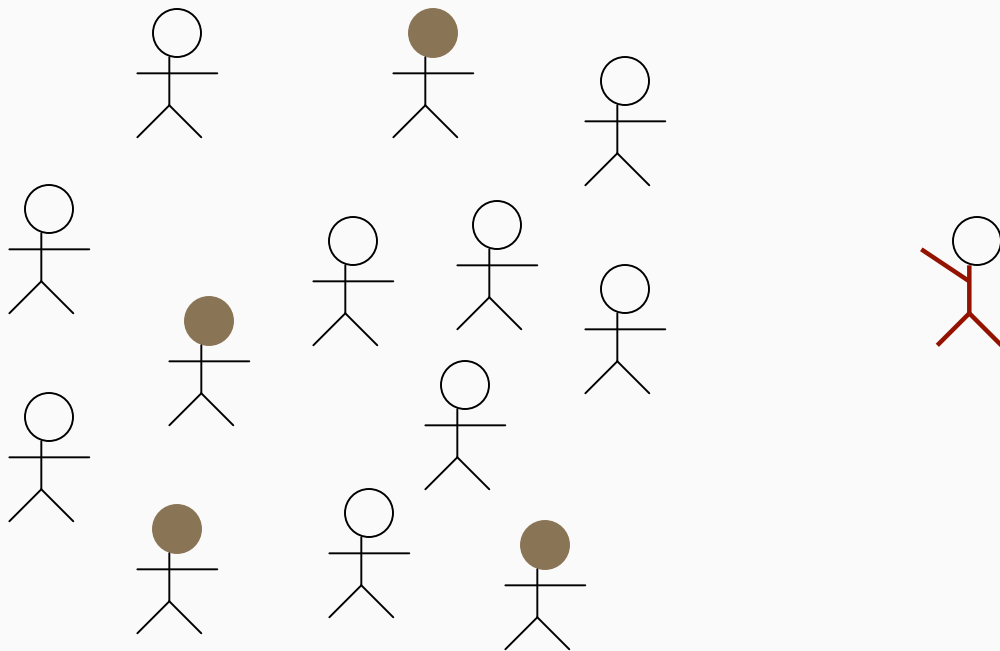


The other didn't go forward ...
I must be dirty too !!!!



$t=2$

In general



If I see k dirty foreheads and they do not go forward all together at time k ,
I go forward at time $k+1$

Some types of knowledge

Topological knowledge

Graph type ("G is a ring"...), adjacency matrix of G ...

Metric knowledge

Number of nodes, diameter, eccentricity...

Sense of direction

Information on link labels

Information on node labels

As the available knowledge grows, the algorithm becomes less portable (rigid). Generic algorithms do not use any knowledge.

Important Facts

State $\times$ Event $\rightarrow$ Action

$\sigma(x, t)$ = internal state of entity x

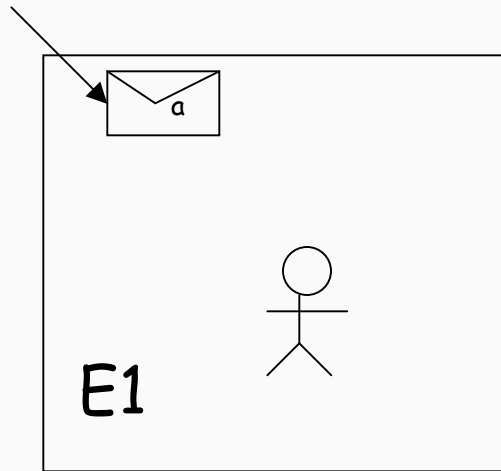
content of memory (registers, clock, ...) at time t

LOCAL INTERNAL STATES

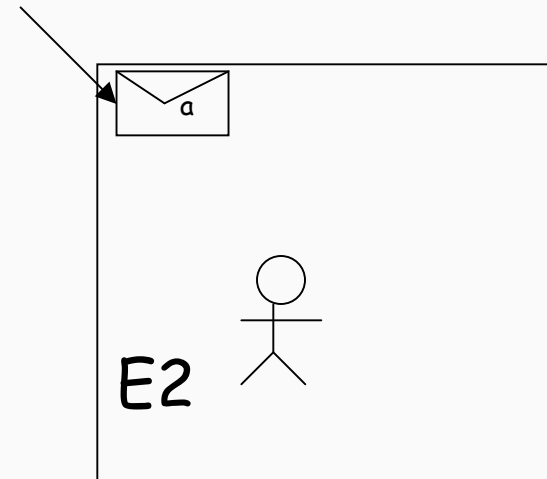
Fact. The same entity x is in 2 distinct environments $E1$ and $E2$. If x is in the same internal state, then she Cannot distinguish between $E1$ and $E2$.

Consequence. Let the same event occur at x . Then, x Will make the same actions and will get the same Internal state.

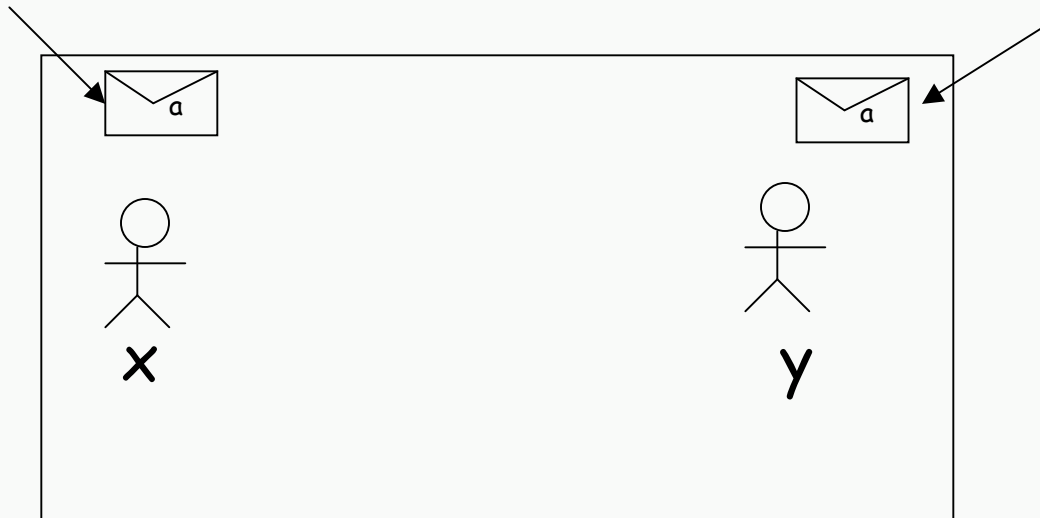
E1, E2: different environments



$$\sigma_1 = \sigma_2$$



1) If the same event happens to x at time t in two different executions and if the internal states σ_1 and σ_2 of x in the two executions at that time are equal, then
the new internal state of x will be the same in both executions



$$\sigma(x) = \sigma(y)$$

2) If the same event happens to x and y at time t in the same execution and if the internal states $\sigma(x)$ and $\sigma(y)$ are equal, then **the new internal states of x and y will be the same.**

Internal states are local and an entity may not
Infer information about the rest of the system

Global State: $\langle \Sigma(t), \text{Future}(t) \rangle$ where

- $\Sigma(t) = \langle \sigma(x,t), \text{for any } x \rangle$
- $\text{Future}(t) = \text{future events generated by } \Sigma(t)$

Ex. $\text{Future}(0) = \text{all spontaneous impulses}$

Goal of Distributed Algorithms:

**Solve Problems, Perform Tasks & Services
In Distributed Systems.**

**Distributed Algorithm: Set of Rules for each
Entity.**

PROBLEMS (TASKS/SERVICES): specify

- 1) Initial Conditions (of the entities)
- 2) Final Conditions (of the entities)
- 3) All Restrictions

$$P = \langle P_{INIT}, P_{FINAL}, R \rangle$$

$P...$ are Predicates on the states of the Entities.

STATUS.

$S = \{\text{all values of the status register}\}$

Protocol B also specifies the value of the status register
At every time t , for any entity x .

$S_I = \text{Initial Status (Start of the Execution of Prot. B)}$

$S_T = \text{Terminal Status (never change status anymore)}$

$S_{\text{start}} < S_I$ (they can start the protocol=Initiators)

$S_{\text{final}} < S_T$ (They can **stop** the execution: do nothing)

TERMINATION.

Protocol B terminates if a $t > 0$ exists s.t. all entities
Enter in a terminal status S_T And $\text{Future}(t) = 0$

Explicit Termination.

All entities enter in a **FINAL STATUS**.

CORRECTNESS.

Protocol B is correct if, starting from any initial Configuration in **Pinit**, it holds

Exists $t > 0$ s.t. **Correct(t)** is true

Where

Correct(t) = for $t' > t$ Predicate **Pfinal** is true.

SOLUTION PROTOCOL.

Protocol B solves problem P if it is correct and
Terminates for any possible initial configuration
And under the restriction R.

Explicit Solution: if B **explicitly terminates**

An example

Back to **Broadcast** ...

Theorem: Under the set of assumptions:

- unique Initiator
- G is connected
- no failures
- bidirectional links

Every generic broadcast protocol requires, in the worst case, m messages.

Lower Bounds for Broadcast

Proof.

$$m(G) = n. \text{ of edges in } G$$

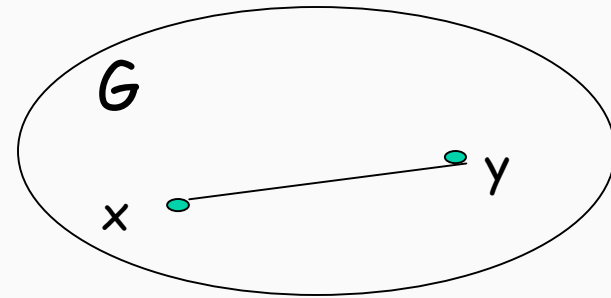
By contradiction.

Let **A** be an algorithm that broadcasts exchanging **less than** $m(G)$ messages (in all executions, and for any graph G) under those assumption.

Then there is at least a link in G where no messages are sent.

Let $e = (x,y)$ be such a link.

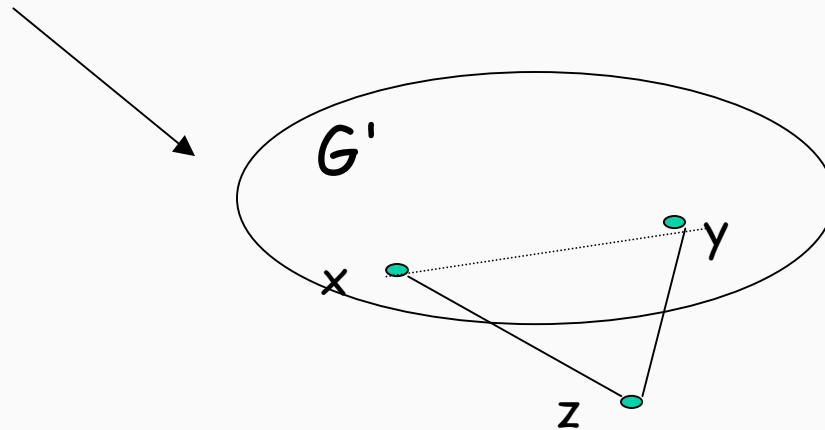
$$G=(V,E)$$



Construct a new graph G'

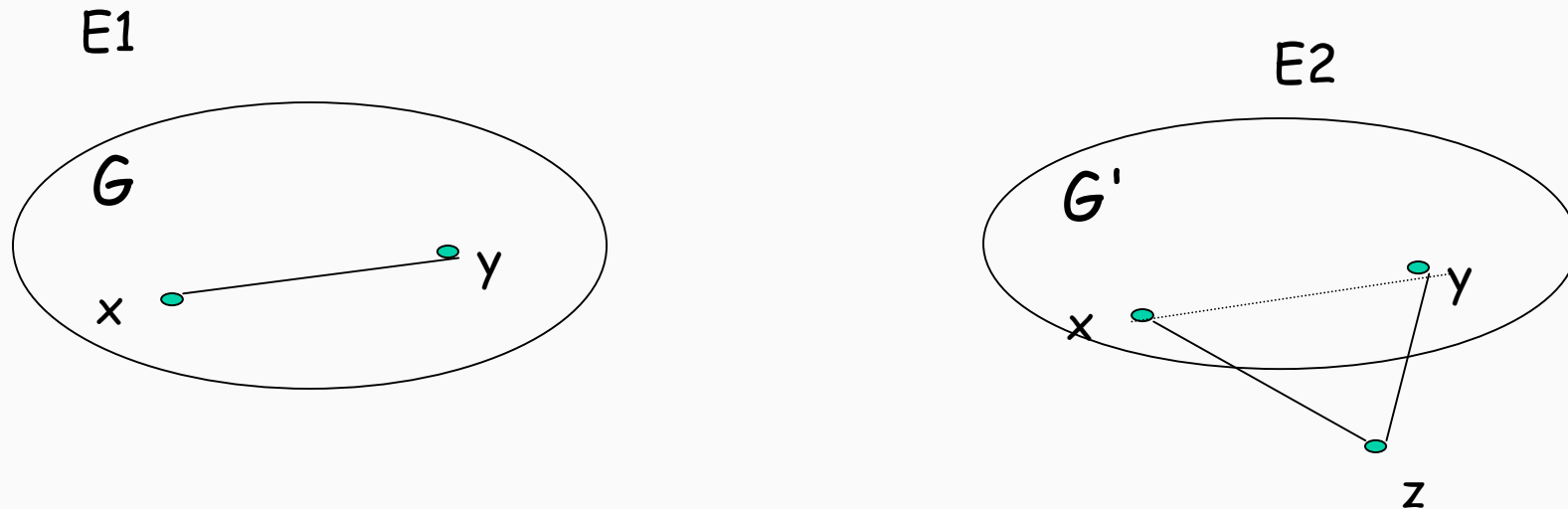
(remember: n is unknown)

$$G' = (V \cup \{z\}, E - e \cup \{(x,z), (y,z)\})$$



Execute the same algorithm on G' with the same time delays, same initial internal states for all nodes except for z which is sleeping

Two executions in two environments



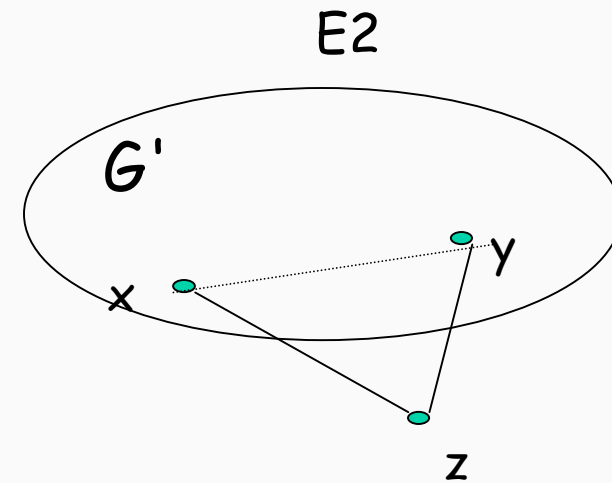
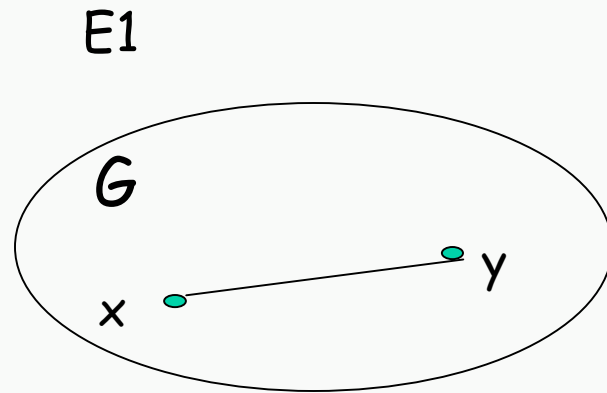
For all nodes, except z, the two executions are **identical**

x and y never send to each other in E1

---->

x and y never send to z in E2

Paola Flocchini



Within finite time the protocol terminates

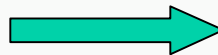
but in E2 node z will never be reached.

Observations:

- 1) Dense networks = more messages
(ex. in complete networks $m = n(n-1) \dots$)
- 2) It is optimum in acyclic graphs

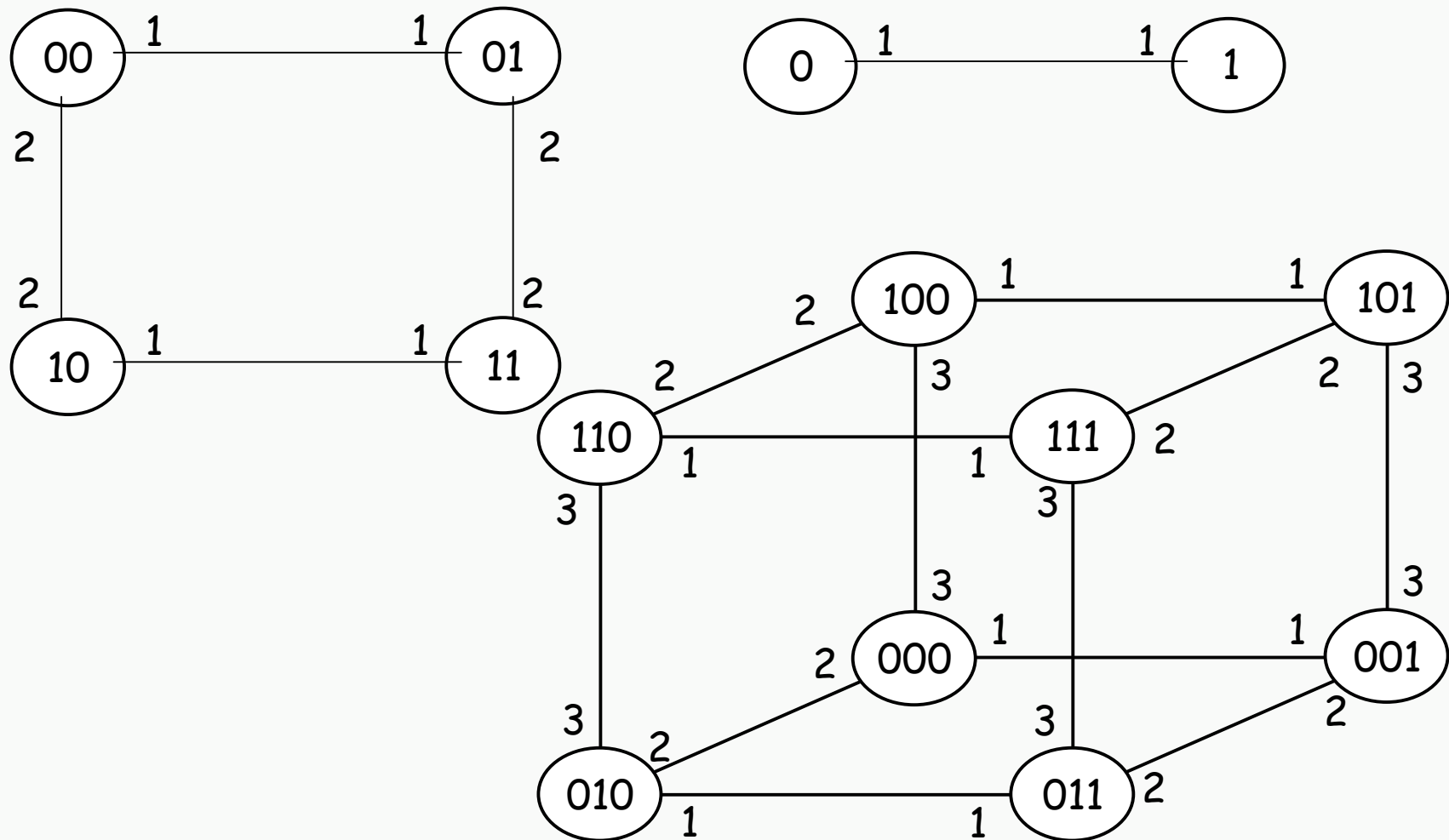
Idea: to solve broadcast.

1. Build a spanning tree of G
2. Execute flooding

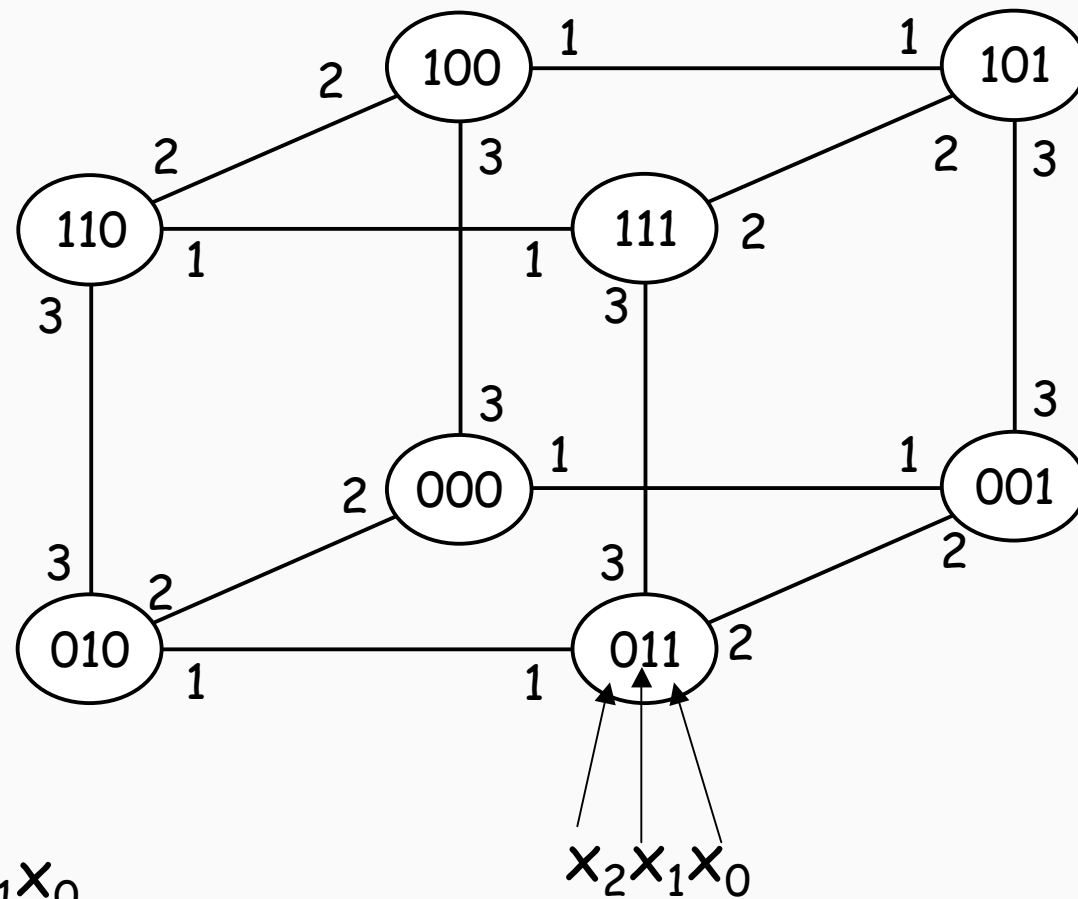


Spanning Tree construction Problem

The Labeled Hypercube



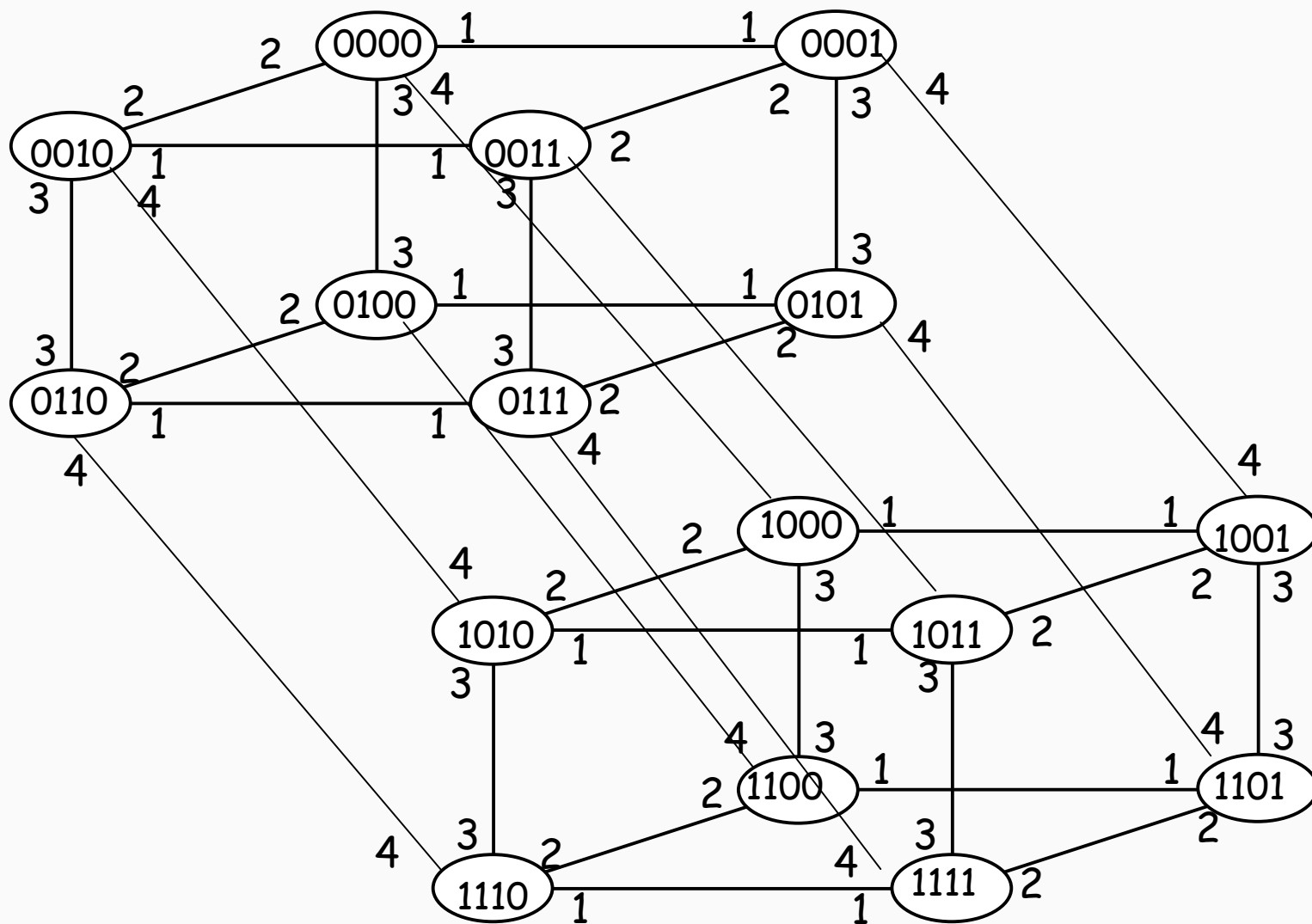
Each link between two nodes is labeled by the dimension of the bit by which the nodes' name differ.



$$X = x_k x_{k-1} \dots x_1 x_0$$

K-bit name

first bit



Paola Flocchini

A hypercube of dimension k has $n = 2^k$ nodes

Each node has k links

$$\rightarrow m = n k / 2 = O(n \log n)$$

Flooding would have message-cost $O(n \log n)$

HyperFlood - Efficient Broadcast

- 1) The initiator sends the message to all its neighbours
- 2) A node receiving the message from link l , sends it only to links with label $l' < l$

Hierarchical Protocol: The Initiator informs one node in each sub-hypercube of dim. $K' < k$. Then msg goes in each of them.

CORRECTNESS

FACT. Every node is touched

Based on the lemma:

For each pair of nodes x and y there exists a path of decreasing labels. **THE MSG WILL BE ROUTED OVER IT.**

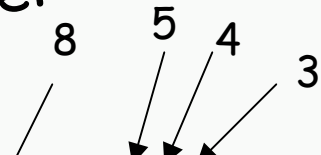
In other words, the used links form a **Spanning Subgraph** of the Hypergraphs.

$$X = x_k, x_{k-1} \dots x_1, x_0$$

$$Y = y_k, y_{k-1} \dots y_1, y_0$$

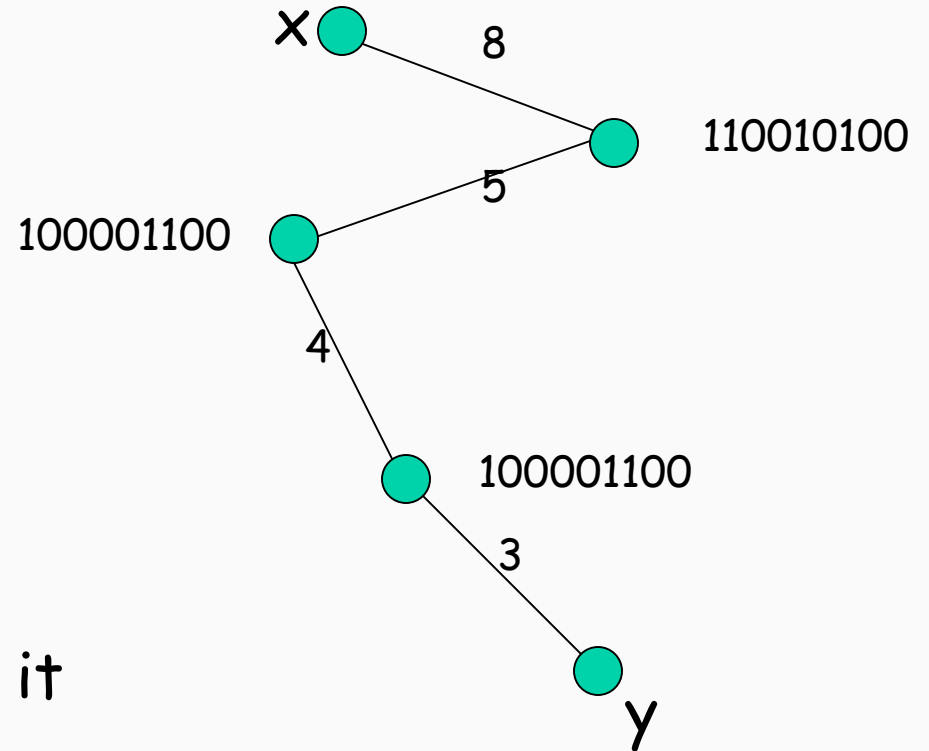
Example:

X= 100010100



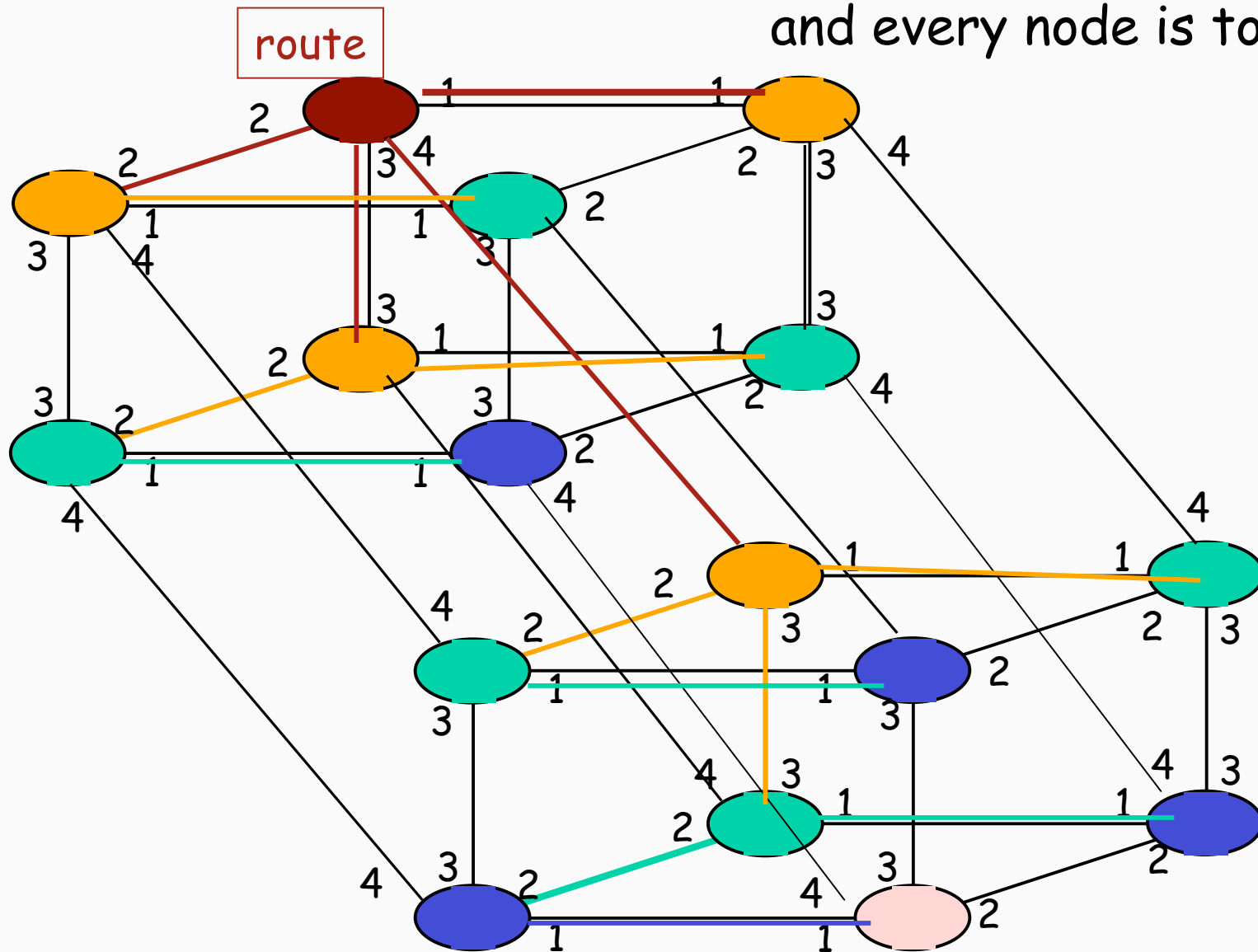
Y= 110001000

2) A node receiving the message from link l , sends it to links with label $l' < l$



y

---> the messages create a spanning tree
and every node is touched



Message Complexity: $n-1$ (OPTIMAL)

Because every entity receives the info only ONCE.

Proof. If not, the **used links** would form
A **CYCLE** --> contradiction with transmissions
Over only decreasing-labels paths.

THM (Ideal TIME). The worst-case is **$\log n$** and It is **OPTIMAL**.

Proof.

- The longest simple path in the induced SPANNER is $\log n =$ the maximal HAMMING Distance between two node-labels.

- At every time-step one new link in the path

Initiator $\text{----> } y$ (for any y) is activated.

Message-Complexity In Special Topologies

General Flooding: $2m - (n-1)$

Ad-hoc algorithm in hypercube: $(n-1)$

Ad-hoc algorithm in complete network: $(n-1)$

In the tree Flooding is optimal: $(n-1)$

WAKE-UP TASK. $V = \text{set of Nodes.}$

Initial Configuration: subset $W \subset V$ are **awake** and $V - W$ are **asleep**. any W -node can start any computation.

Final Configuration: All nodes in V must be awaked.

Standard Restrictions: $BL, CN, TR = R$

Protocol WFLOOD.

ASLEEP

External Event (Spontaneously)
send(wake-up!) to $N(x)$
become AWAKE

External Event Receiving(wake-up!)
send(wake-up!) to $N(x) - \{\text{sender}\}$
become AWAKE

EXERCISE: Prove its Correctness.

WAKE-UP TASK

General Topologies.

Protocol WFLOOD:

$$2^{m-n+1} \leq M(\text{WFLOOD}) \leq 2^m \quad (\text{Optimal})$$

$$\text{Time}(\text{WFLOOD}) = D \quad (\text{Optimal})$$

Trees.

$$M(\text{Wflood/Tree}) = n + k^* - 2 \quad (1)$$

$k^* = \# \text{ Initiators}$

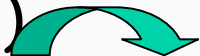
Exercise: Prove (1)

Computations with Multiple initiator: WAKE-UP

COMPLETE GRAPH

Broadcast

Flood	Specific
$O(m=n^2)$	$O(n)$



Wakeup

Flood	Specific
	$\Omega(n^2)$

Need additional assumptions
to reduce the complexity

HYPERCUBE

Broadcast

Flood	Specific
$O(m=n \log n)$	$O(n)$



Wakeup

Flood	Specific
	$\Omega(n \log n)$

Complete Graphs with Unique Labels (ID's)

Restriction. Each node has a unique name.

Thm. $M(\text{Wake-up}/K, \text{Id}) = O(n \log n)$

The protocol is complex and solves the *Harder* problem of ELECTION.

Can we do better?

Answer: NO !!

THM. $M(\text{Wake-up} / K, ID) = \Omega(n \log n)$

Proof.

Proving a lower bound in Distributed Computing
is a 2-player **GAME**:

Protocol Prot vs **Adversary ADV**

Prot is fixed

ADV knows it.

ADV, looks at Prot, and decides:

- 1. the node labels (distinct)
- 2. Which Initiators andWhen they start
- 3. Message delays
- 4. matching links $\leftrightarrow$ port-labels (for any sender)!!

Key-Idea: ADV maximizes # of *useless* msgs among AWAKED NODES

TOOLS: (3) and (4).

Def. node subsets A and B are *connected* at time t if a in A and b in B exist s.t. a and b exchange a msg at time t

$t=1$ ADV wakes-up the seed s . When s sends $m(s)$, ADV sets label l to edge e to y .

(3) y does not receive $m(s)$ until y sends $m(y)$ on l' .
--> ADV assigns l' to e .

So, ADV let **s** and **y** to receive the msgs at the same time when they are **ALREADY AWAKED**.

NOTES. Sending msgs to more than 1 port is equivalent to send the msgs to each port, one at the time.

The node (i.e. PROT) sees only its ports with their labels and not the topological links.

General Adv's Rules

A = { AWAKED-NODES }

ADV's Goal: msgs are sent to AWAKED nodes and
A is always connected

When an AWAKED node selects an USED port then
ADV *does nothing*....since the msg is useless...

WHILE....

Let x in A be a node running $\text{send}(m(x))$ on **unassigned port-label** a . Then, ADV distinguishes two cases:

i) x has a free link e connecting an **AWAKE** node.

Then ADV assign $a \rightarrow e$

ii) No free links. Create a new node-subset B AND then connect it to A (New **STAGE**)

Let's see how. Let $k = \# \text{ AWAKE NODES}$

$A = \{x_0=s, x_1, \dots, x_k\}$ (Ordered by Wake-up Time)

create set of new nodes

$B = \{z_0, z_1, \dots, z_k\}$ so $|A| = |B|$

SET $x=x_i$ for some i . Then $m(x_i)$ is sent to z_i

ADV makes the History of B independent and identical to that of A ($x_i \approx z_i$):

For each $i=1,\dots,k$: z_i will behave exactly as x_i

but... when z_i has used all its B -links (Phase 1), then

the msg $m(x_i)$ sent to z_i is **delayed** until z_i wants to send $m(z_i)$ over a **free** label and **all** B -links have been used. At that time (Phase 2), ADV assigns the free label to link $z_i \leftrightarrow x_i$

IMPORTANT.

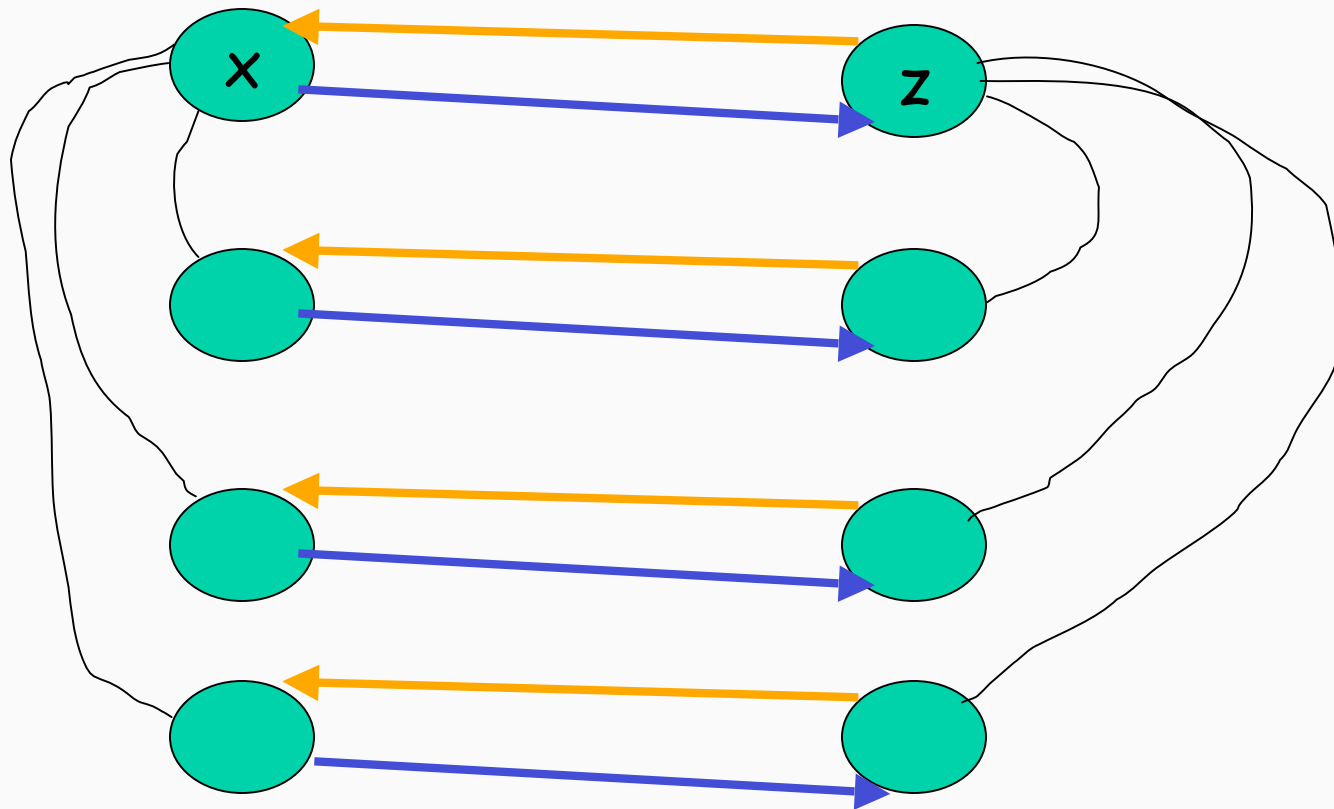
The behaviour of B is independent and identical to A since there is no A→B msg-reception before the end of Phase 1.

A

B

The Adv forms a seq.
of **Layers**

Phase 2



Phase 1

Cost Analysis.

Def. $\text{Active}(i) = \{\text{Awake nodes at stage } i\}$

$$\text{New}(i) = \text{Active}(i) - \text{Active}(i-1)$$

Fact 1. $\text{Active}(0) = \{s\}$ and $|\text{New}(i)| = |\text{Active}(i-1)|$

Def. $M(i-1) = \#$ msgs exchanged before Stage i

Fact 2. ADV forces PROT (i.e. the active nodes) to let the new nodes to exchange at least the same $\#$ of msgs of Stage $(i-1)$ before connecting A to B , i.e.

$$\# \text{ of msgs} = M(i-1)$$

Consequence. Total $\#$ msgs BEFORE $A-B$ connection at Stage i is at least

$$\begin{array}{ccccc} M(i-1) & + & M(i-1) & = & 2 M(i-1) \\ \text{Stage } i-1 & & \text{Stage } i \text{ before } A-B & & \end{array}$$

Main Question: How many msgs BEFORE stage $i+1$?
(need analysis of Phase 2)

The exact # depends on Prot BUT we can lower bound.

Obs. ADV does not start a new STAGE until

* There is x in $\text{Active}(i)$ sending $m(x)$ on a new label /

AND

** All links from x to $\text{Active}(i)$ are not free

Consequence. x has communicated with all nodes in

$$\text{Active}(i) = \text{Active}(i-1) \cup \text{New}(i)$$

Two cases:

- x in $\text{Active}(i-1)$. Then, msgs $x \longleftrightarrow \text{New}(i)$ happens in Stage i

So, **after** connection (Phase 2)

$$\#msg(i) \geq |\text{New}(i)| = |\text{Active}(i-1)|$$

- x in $\text{New}(i)$. Then ALL its msgs are at STAGE i
again,

$$\#msg(i) \geq |\text{New}(i)| = |\text{Active}(i-1)|$$

Combining

$$M(i) \geq 2 M(i-1) + |\text{Active}(i-1)|$$

Phase 1

Phase 2

$$|\text{Active}(0)| = 1 \text{ and } |\text{Active}(i)| = 2 |\text{Active}(i-1)|$$

so $|\text{Active}(i)| = 2^i$

$$\begin{aligned} M(i) &\geq 2 M(i-1) + 2^{i-1} \geq^* i \cdot 2^{i-1} = \\ &= \Omega(n \log n) \end{aligned}$$

Since $\# \text{ STAGES} = \log n$

* Exercise